Linux Command Line Guide

Essential commands for working on your C project in Ubuntu (WSL)

How to read a prompt

```
yourname@YOURPC:~/forex-project$
```

- **yourname@YOURPC**: your Linux user and the computer name.
- **~/forex-project**: the folder you are in. The **~** symbol means your home folder.
- **$**: the shell is ready for a command. Do not type the $ yourself.

A command usually looks like: **command -options arguments**, for example **ls -l src**.

Note: Linux is case-sensitive. **Forex** and **forex** are different names, and commands must be typed in lowercase.

1. Moving around folders

Linux uses **cd** (change directory), the same as Windows cmd, but with **/** instead of ****.

I want to...	Command
Go into a folder	`cd forex-project`
Go back up one folder (leave the current one)	`cd ..`
Go up two folders	`cd ../..`
Go to your home folder	`cd ~ (or just: cd)`
Go back to the previous folder	`cd -`
Go to an exact location	`cd /home/yourname/forex-project`
Show where I am	`pwd`
List what is in this folder	`ls`
List with details (size, date, permissions)	`ls -l`
Include hidden files (names starting with .)	`ls -a`
Details, hidden files, and readable sizes	`ls -lah`

Special folder names:

- **.** means the current folder, and **..** means the parent folder (one level up).
- **~** means your home folder, and **/** is the very top of the filesystem.
- Your Windows C: drive appears at **/mnt/c**.

Exiting folders vs. exiting Linux: use **cd ..** to go up out of a folder. The **exit** command leaves Linux entirely and returns you to Windows.

2. Creating, copying, moving and deleting

I want to...	Command
Create a folder	`mkdir name`

I want to...	Command
Create nested folders in one go	`mkdir -p project/src/tests`
Create an empty file	`touch notes.txt`
Copy a file	`cp a.c b.c`
Copy a folder and everything in it	`cp -r src src_backup`
Move a file to another folder	`mv a.c src/`
Rename a file	`mv old.c new.c`
Delete a file	`rm file.c`
Delete an empty folder	`rmdir name`
Delete a folder and all its contents	`rm -r name`

Note: Deleting is permanent. There is no recycle bin in the terminal. Double-check before using **rm -r**, and never run **rm -rf /** or anything similar. Use **rm -i file** to be asked for confirmation first.

3. Looking at file contents

I want to...	Command
Print a whole file	`cat file.c`
Print with line numbers	`cat -n file.c`
Scroll through a long file (q to quit)	`less file.c`
Show the first 10 lines	`head data.csv`
Show the first 5 lines	`head -n 5 data.csv`
Show the last 10 lines	`tail data.csv`
Count lines, words and bytes	`wc data.csv`
Count only lines (number of rows)	`wc -l data.csv`

Useful for this project: before loading a forex CSV in C, run **head data.csv** to see the column layout and **wc -l data.csv** to see how many rows it has.

4. Editing files with nano

nano is a simple terminal text editor, good for quick edits. (For real coding, use VS Code with the WSL extension.)

```
nano hello.c
```

I want to...	Keys / command
Save the file	`Ctrl+O, then Enter`
Exit nano	`Ctrl+X`
Search for text	`Ctrl+W`
Cut the current line	`Ctrl+K`

I want to...	Keys / command
Paste the cut line	`Ctrl+U`
Open the current folder in VS Code	`code .`

When you exit with unsaved changes, nano asks whether to save. Press **Y** to save or **N** to discard.

5. Compiling and running C programs

I want to...	Command
Compile a single file	`gcc hello.c -o hello`
Compile with warnings (recommended)	`gcc -Wall -Wextra hello.c -o hello`
Compile with debug info	`gcc -Wall -Wextra -g hello.c -o hello`
Compile with memory-error detection	`gcc -Wall -Wextra -g -fsanitize=address,undefined hello.c -o hello`
Compile several files together	`gcc -Wall -Wextra main.c indicators.c -o backtest`
Link the math library (needed for sqrt, pow, log)	`gcc main.c -o app -lm`
Run a program in the current folder	`./hello`
Build using a Makefile	`make`
Remove build files (if your Makefile has a clean rule)	`make clean`
Check for memory leaks	`valgrind --leak-check=full ./hello`
Debug with gdb	`gdb ./hello`

Note: You must write **./** before the program name to run it. Linux does not search the current folder by default, so typing just **hello** gives "command not found".

6. Finding things

I want to...	Command
Search for text inside files	`grep "malloc" *.c`
Search a whole folder, with line numbers	`grep -rn "malloc" .`
Search ignoring upper/lower case	`grep -i "error" log.txt`
Find files by name	`find . -name "*.c"`
Show where a command lives	`which gcc`
Read the manual for a command	`man printf`
Quick help for most commands	`gcc --help`

In the manual pages (**man**), use the arrow keys to scroll and press **q** to quit. Section 3 covers C library functions, so **man 3 malloc** shows the C function documentation.

7. Combining commands

The pipe symbol **|** sends the output of one command into another. The symbols **>** and **>>** save output to a file.

```
ls -l | less                  # scroll a long listing
grep -c "EUR" data.csv        # count matching lines
./backtest > results.txt      # save output (overwrites file)
./backtest >> results.txt     # append to the file
gcc main.c -o app && ./app    # run only if compile succeeds
```

Text after # is a comment, so you can leave it out when typing.

8. Permissions, sudo and installing software

I want to...	Command
Run a command as administrator	`sudo command`
Refresh the list of available packages	`sudo apt update`
Install a package	`sudo apt install package-name`
Upgrade installed packages	`sudo apt upgrade`
Make a script runnable	`chmod +x script.sh`
See your username	`whoami`

sudo asks for your Linux password (nothing shows as you type, which is normal) and remembers it for about 15 minutes. You do not need **sudo** to compile or run your own programs.

9. Keyboard shortcuts that save time

I want to...	Shortcut
Auto-complete file and folder names	`Tab (press twice to list options)`
Previous commands	`Up arrow / Down arrow`
Search your command history	`Ctrl+R, then type`
Stop a running program	`Ctrl+C`
Clear the screen	`Ctrl+L (or: clear)`
Jump to start / end of the line	`Ctrl+A / Ctrl+E`
Leave Linux (back to Windows)	`exit (or Ctrl+D)`
Paste into the terminal	`Ctrl+Shift+V (or right-click)`

Ctrl+C is important: if your program is stuck in an infinite loop, it stops it. **Tab** is the biggest time saver. Type the first few letters of a name and press it.

Practice session

Type these in order to try everything above:

```
cd ~
```

```
mkdir -p forex-project/src
cd forex-project/src
pwd
nano hello.c          # type a small program, save, exit
cat -n hello.c
gcc -Wall -Wextra hello.c -o hello
./hello
cd ..
ls -l
cd ~
```

Next step: create **indicators.h** and **indicators.c** with a simple moving average (SMA) function, and a small test program to check it.